Redução de dimensionalidade

Prof. Aldebaro Klautau
Universidade Federal do Pará (UFPA)

www.lasse.ufpa.br & www.ppgee.ufpa.br

Projection

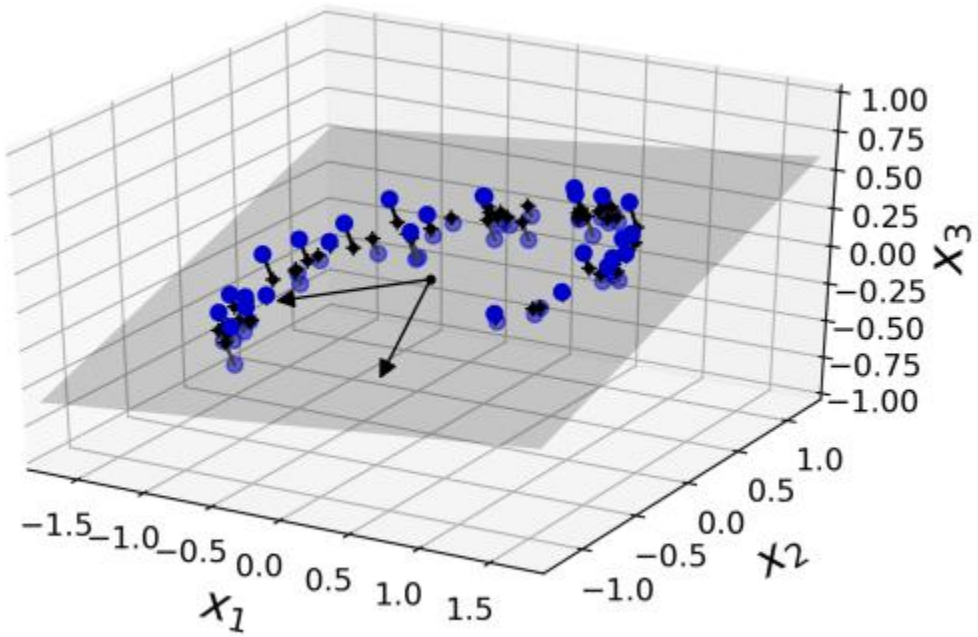


Figure 8-2. A 3D dataset lying close to a 2D subspace

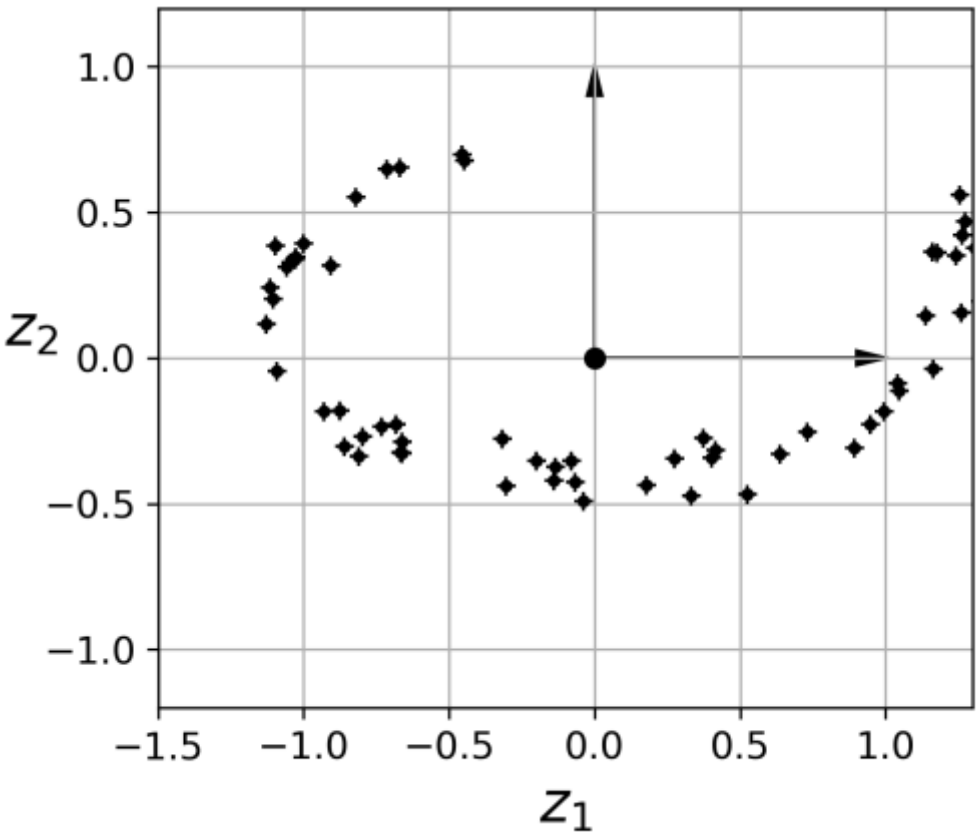


Figure 8-3. The new 2D dataset after projection

Projections may not work

- The Swiss roll is an example of a 2D **manifold**. Put simply, a 2D manifold is a 2D shape that can be bent and twisted in a higher-dimensional space. More generally, a d-dimensional manifold is a part of an n-dimensional space (where d < n) that locally resembles a d-dimensional hyperplane. In the case of the Swiss roll, d = 2 and n = 3: it locally resembles a 2D plane, but it is rolled in the third dimension.

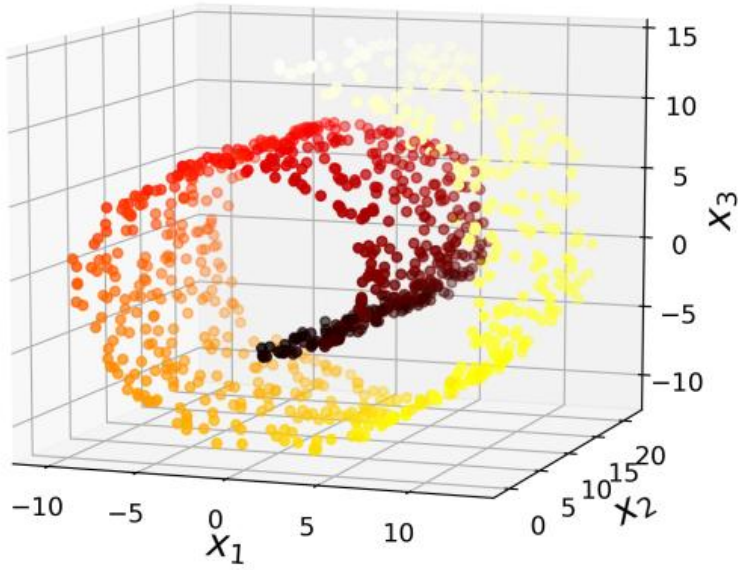


Figure 8-4. Swiss roll dataset

Simply projecting onto a plane (e.g., by dropping x_3) would squash different layers of the Swiss roll together, as shown on the left side of Figure 8-5. What you really want is to unroll the Swiss roll to obtain the 2D dataset on the right side of Figure 8-5.

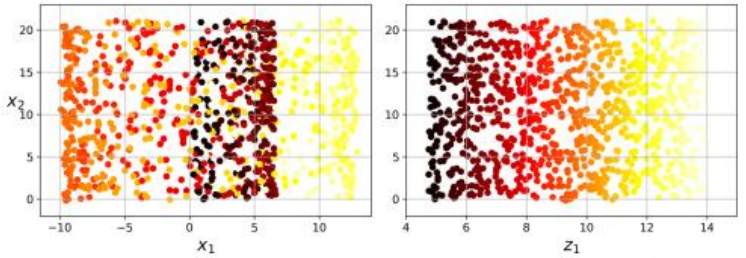


Figure 8-5. Squashing by projecting onto a plane (left) versus unrolling the Swiss roll (right)

https://bjlkeng.github.io/posts/manifolds/

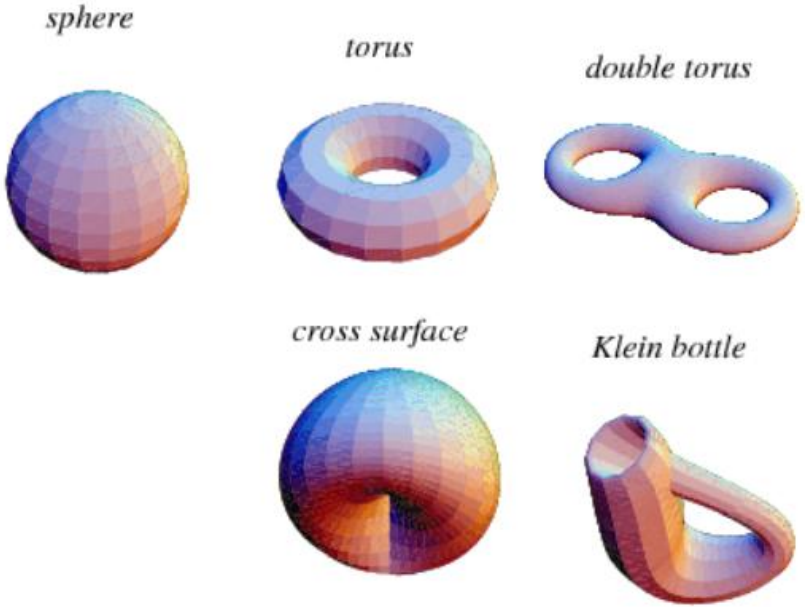


Figure 3: Non-intersecting closed surfaces in $\mathbb{R}^3$ are examples of 2D manifolds such as a sphere, torus, double torus, cross surfaces and Klein bottle (source: Wolfram).

Principal component analysis (PCA)

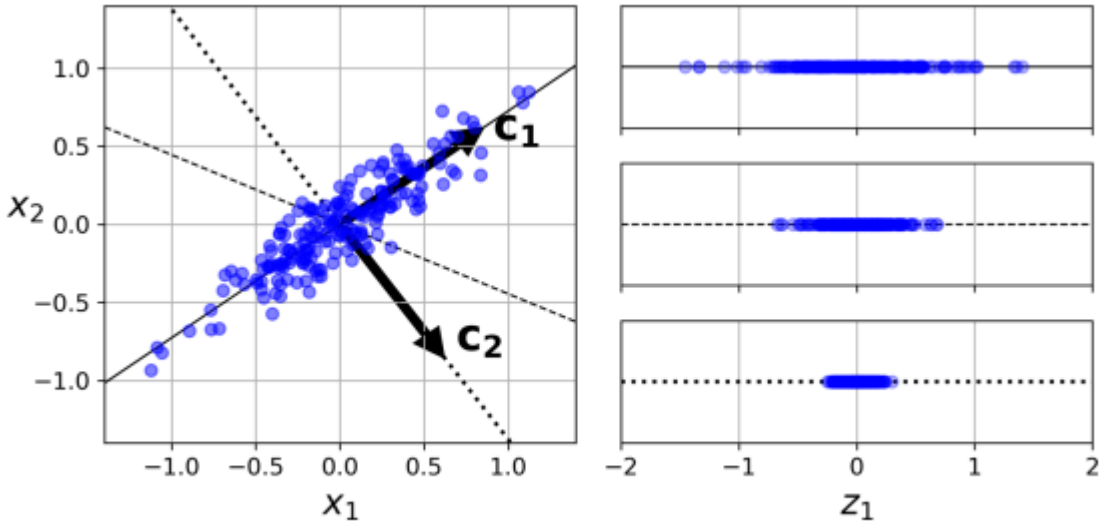


Figure 8-7. Selecting the subspace to project on

Matrix multiplication is a linear transform.
PCA is a linear transform using a specific matrix

In linear algebra, any linear transformation[1] (or transform) can be represented by a matrix **A**. The linear transform operation is given by

$$\mathbf{y} = \mathbf{A}\mathbf{x} \tag{2.1}$$

where **x** and **y** are the input and output column vectors, respectively. For example, the matrix

$$\mathbf{A} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \tag{2.2}$$

corresponds to a clockwise rotation of the input vector by an angle θ. Figure 2.1 illustrates the rotation for a vector $\mathbf{x} = [4, 8]^T$ by an angle $\theta = \pi/2$ radians, i. e., $\mathbf{A} = [0, 1; -1, 0]$, resulting in $\mathbf{y} = \mathbf{A}\mathbf{x} = [8, -4]^T$.

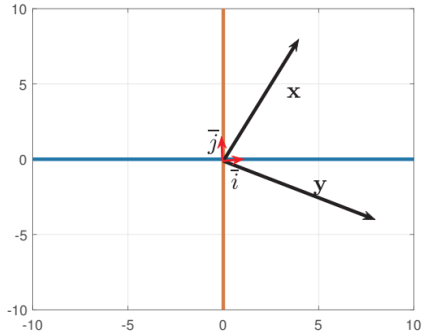


Figure 2.1: *Rotation of a vector* $\boldsymbol{x}$ *by an angle* $\theta = \pi/2$ *radians using* $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$ *with* $\boldsymbol{A}$ *given by Eq. (2.2).*

A.15.2 Projection of a vector using inner product

To explore the properties and advantages of linear transforms, it is useful to study the *vector projection* (or simply projection) of a vector onto another one.

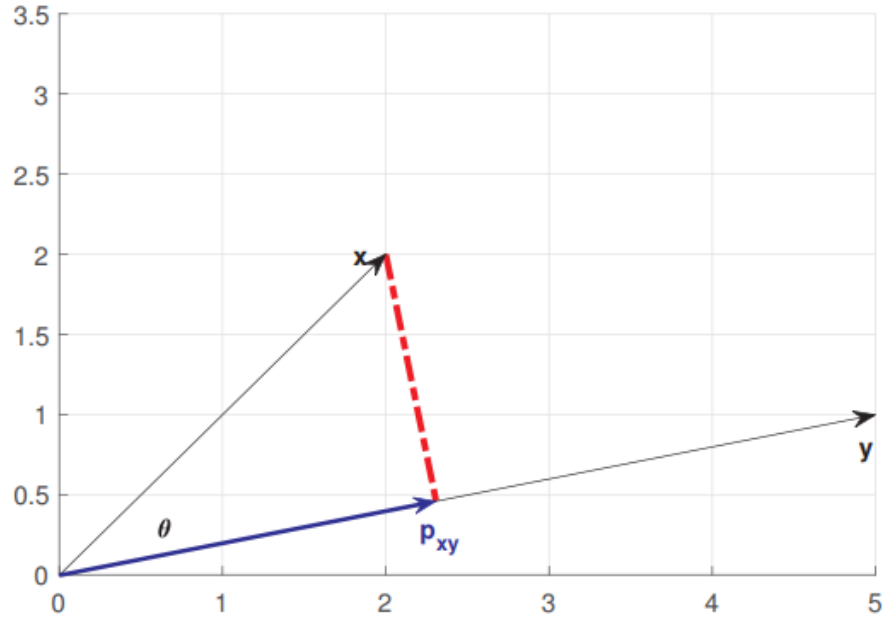


Figure A.2: *The perpendicular line for obtaining the projection* $\boldsymbol{p}_{xy}$ *of a vector* $\boldsymbol{x}$ *onto* $\boldsymbol{y}$ *in* $\mathbb{R}^2$. *Note that* $\theta = \cos^{-1}(\langle \boldsymbol{x}, \boldsymbol{y} \rangle / (\|\boldsymbol{x}\| \, \|\boldsymbol{y}\|)$ *is the angle between* $\boldsymbol{x}$ *and* $\boldsymbol{y}$ *and the inner product* $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \|\boldsymbol{p}_{xy}\| \, \|\boldsymbol{y}\| = \|\boldsymbol{p}_{yx}\| \, \|\boldsymbol{x}\|$.

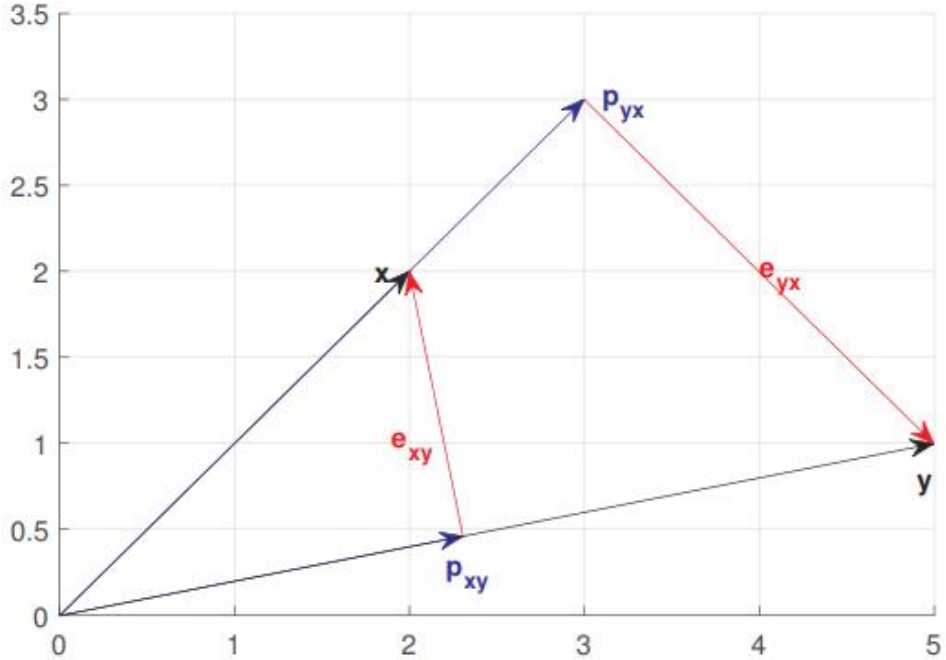
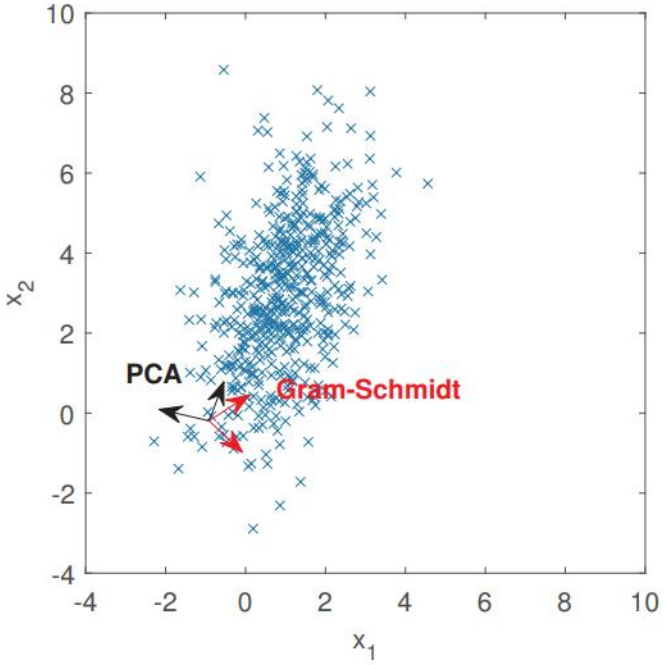
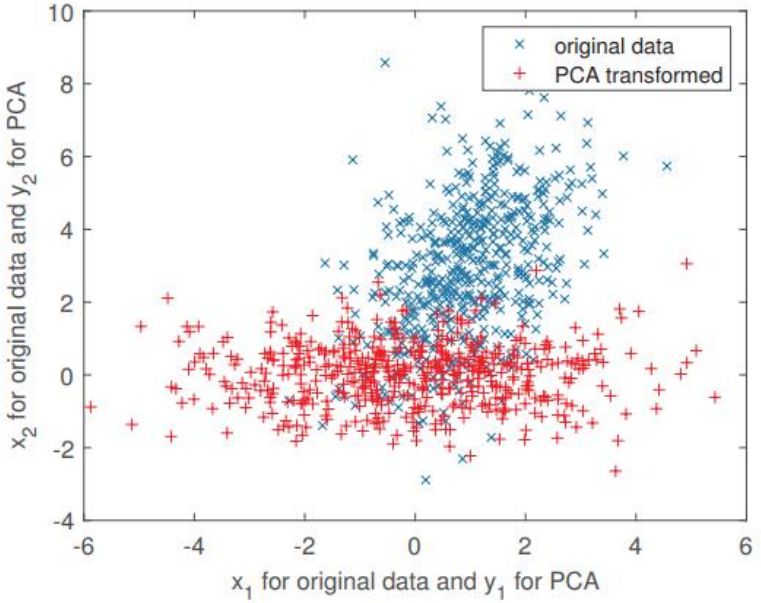


Figure A.3: *Projections of a vector* $\boldsymbol{x}$ *and* $\boldsymbol{y}$ *onto each other. Note the errors are orthogonal to the directions of the respective projections.*

Example of PCA





Using Scikit-Learn

Scikit-Learn's PCA class uses SVD decomposition to implement PCA, just like we did earlier in this chapter. The following code applies PCA to reduce the dimensionality of the dataset down to two dimensions (note that it automatically takes care of centering the data):

```
from sklearn.decomposition import PCA

pca = PCA(n_components = 2)
X2D = pca.fit_transform(X)
```

After fitting the PCA transformer to the dataset, its components_ attribute holds the transpose of $\mathbf{W}_d$ (e.g., the unit vector that defines the first principal component is equal to pca.components_.T[:, 0]).

Explained Variance Ratio

Another useful piece of information is the *explained variance ratio* of each principal component, available via the `explained_variance_ratio_` variable. The ratio indicates the proportion of the dataset's variance that lies along each principal component. For example, let's look at the explained variance ratios of the first two components of the 3D dataset represented in Figure 8-2:

```
>>> pca.explained_variance_ratio_
array([0.84248607, 0.14631839])
```

This output tells you that 84.2% of the dataset's variance lies along the first PC, and 14.6% lies along the second PC. This leaves less than 1.2% for the third PC, so it is reasonable to assume that the third PC probably carries little information.

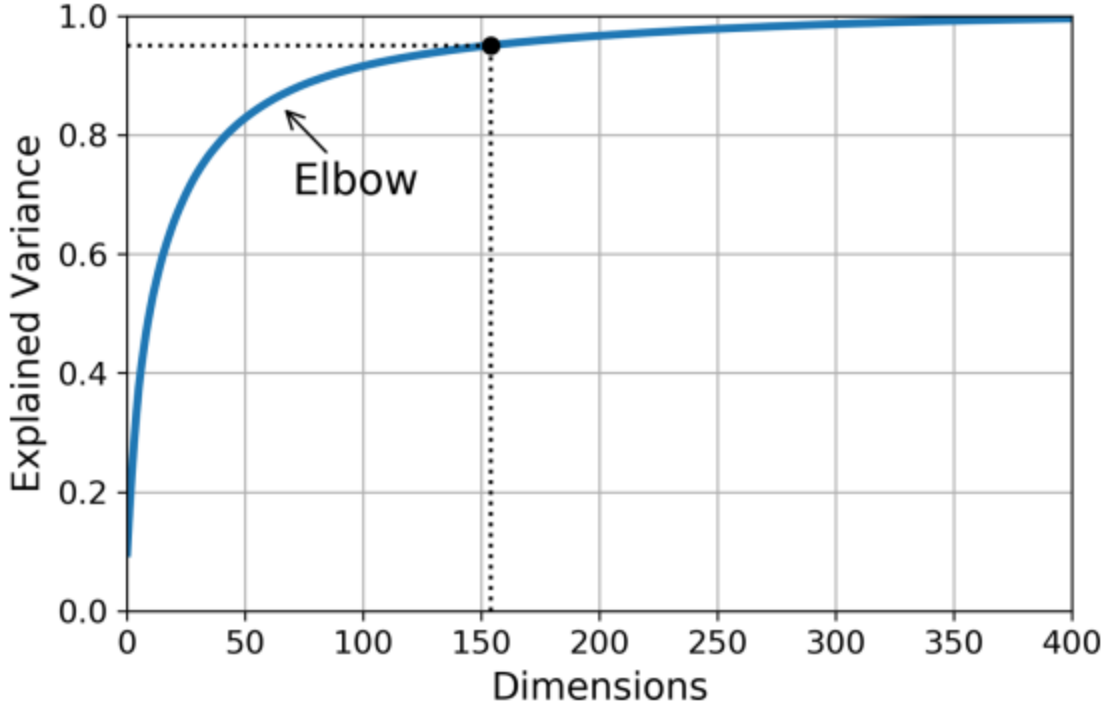


Figure 8-8. Explained variance as a function of the number of dimensions

Choosing the Right Number of Dimensions

Instead of arbitrarily choosing the number of dimensions to reduce down to, it is simpler to choose the number of dimensions that add up to a sufficiently large portion of the variance (e.g., 95%). Unless, of course, you are reducing dimensionality for data visualization—in that case you will want to reduce the dimensionality down to 2 or 3.

The following code performs PCA without reducing dimensionality, then computes the minimum number of dimensions required to preserve 95% of the training set's variance:

```
pca = PCA()
pca.fit(X_train)
cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1
```

You could then set `n_components=d` and run PCA again. But there is a much better option: instead of specifying the number of principal components you want to preserve, you can set `n_components` to be a float between 0.0 and 1.0, indicating the ratio of variance you wish to preserve:

```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
```

The following code compresses the MNIST dataset down to 154 dimensions, then uses the `inverse_transform()` method to decompress it back to 784 dimensions:

```
pca = PCA(n_components = 154)
X_reduced = pca.fit_transform(X_train)
X_recovered = pca.inverse_transform(X_reduced)
```

Figure 8-9 shows a few digits from the original training set (on the left), and the corresponding digits after compression and decompression. You can see that there is a slight image quality loss, but the digits are still mostly intact.

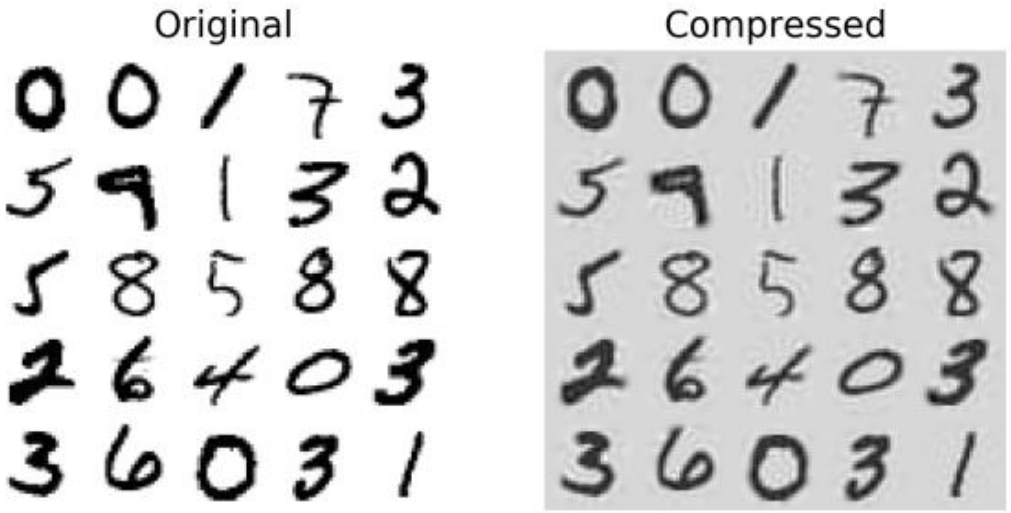


Figure 8-9. MNIST compression that preserves 95% of the variance

PCA for compression

- Other PCA techniques:
  - Incremental PCA
  - Kernel PCA (related to kernel in SVM)

Other dimensionality reduction techniques

t-Distributed Stochastic Neighbor Embedding (t-SNE)

Reduces dimensionality while trying to keep similar instances close and dissimilar instances apart. It is mostly used for visualization, in particular to visualize clusters of instances in high-dimensional space (e.g., to visualize the MNIST images in 2D).

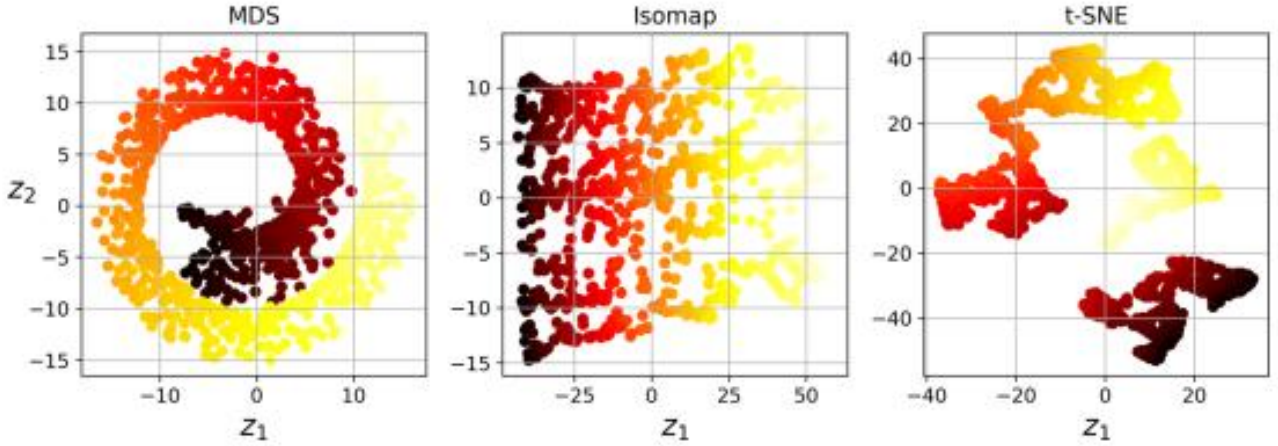


Figure 8-13. Using various techniques to reduce the Swill roll to 2D